

Sí, puedes copiar objetos entre formularios en Oracle Forms

Tu idea es **válida y es una práctica común** cuando un formulario nuevo no se integra bien al menú del sistema. Vamos por partes.

Por qué probablemente no funciona tu formulario nuevo en AcseIX

Antes de copiar, entiende la causa. Cuando un formulario nuevo no se abre desde el menú de un sistema existente, casi siempre es por uno de estos motivos:

Causa	Cómo verificarla
El <code>.fmx</code> no está en el <code>FORMS_PATH</code>	El formulario se compila pero el servidor no lo encuentra al llamarlo
El menú apunta a una versión antigua	El <code>.mmx</code> no se recompiló después de agregar la opción
Ruta relativa vs absoluta	<code>CALL_FORM('FOR_BLOQUEO_PV')</code> requiere que esté en el <code>FORMS_PATH</code>
Falta la librería de AcseIX	El formulario de AcseIX usa librerías (<code>.p11</code>) que el tuyo no tiene adjuntas
Falta la conexión a BD de AcseIX	El <code>WHEN-NEW-FORM-INSTANCE</code> de AcseIX establece variables globales (<code>:GLOBAL.empresa</code> , <code>:GLOBAL.usuario</code> , etc.) que tu form no tiene
El formulario no está registrado en la tabla de formularios de AcseIX	Algunos sistemas tienen un catálogo de formularios permitidos

Antes de copiar todo, revisa esto: si el problema es solo la ruta o el registro en el menú, **no necesitas copiar nada**, solo arreglar el despliegue.

✅ ¿Se pueden copiar objetos entre formularios? Sí

Oracle Forms permite **copiar y pegar objetos** entre dos módulos abiertos simultáneamente. Puedes copiar:

- Bloques de datos (BLK_PV, BLK_FILTRO, etc.)
- Bloques de control (BLK_BOTONES, BLK_ARREGLO)
- Canvas (CVS_LISTA, CVS_ARREGLO)
- Ventanas (WIN_MAIN, WIN_ARREGLO)
- Ítems
- Triggers
- Program Units
- LOVs y Record Groups
- Alertas

NO se copian automáticamente:

- Librerías adjuntas (.pll) → hay que adjuntarlas manualmente
 - Objetos de base de datos (tablas, secuencias, vistas) → ya deben existir en el esquema
 - Parámetros de módulo → hay que revisarlos
-

⚠️ Antes de copiar: entiende el riesgo

Copiar TODO el formulario puede ser problemático por estas razones:

- 1.**Nombres duplicados:** si el form destino ya tiene un bloque llamado BLK_ACCION o un ítem ID_PUNTO_VENTA, el pegado fallará o creará conflictos.
 - 2.**Variables globales:** si el form de AcseIX ya usa :GLOBAL.usuario, :GLOBAL.empresa, etc., tu código que también las usa puede sobrescribirlas o leer valores distintos.
 - 3.**Librerías adjuntas:** el form destino tiene librerías de AcseIX (funciones comunes). Tu código debe usar las mismas o convivir con ellas.
 - 4.**WHEN-NEW-FORM-INSTANCE:** no puedes tener dos. Debes fusionar el tuyo con el del form destino.
 - 5.**Menú adjunto:** si el form destino tiene menú propio (MENU_PRINCIPAL de AcseIX), tu form de bloqueo no debería traer uno.
-



Guía paso a paso para copiar objetos

Paso 1: Abre ambos formularios al mismo tiempo

1. Abre el formulario **destino** (el que ya funciona en AcselX).
2. Abre el formulario **origen** (FOR_BLOQUEO_PV.fmb) en otra ventana de Forms Builder.
3. Ambos aparecen en el **Object Navigator**. Puedes tener 2 ventanas del Object Navigator abiertas al mismo tiempo (una por cada módulo).

Paso 2: Prepara el formulario destino

1. **Identifica los 3 inputs** que quieres quitar. Anota su bloque y canvas.
2. Si esos 3 inputs son el único contenido de un bloque, elimina el bloque completo. Si están mezclados con otros objetos del form de AcselX, elimínalos individualmente.
3. **Anota los nombres** de todos los bloques, ítems, canvas y ventanas que ya existen en el form de AcselX. Los necesitarás para evitar conflictos.

Paso 3: Copia los objetos del formulario origen

En el **Object Navigator** del formulario origen (FOR_BLOQUEO_PV):

1. Copia los bloques de datos:

- Selecciona BLK_PV, BLK_FILTRO, BLK_BOTONES, BLK_ARREGLO (con Ctrl para seleccionar varios).
- Presiona **Ctrl + C**.

2. Copia las ventanas y canvas:

- Selecciona WIN_MAIN, WIN_ARREGLO (Ctrl).
- Presiona **Ctrl + C**.
- Selecciona CVS_LISTA, CVS_ARREGLO (Ctrl).
- Presiona **Ctrl + C**.

3. Copia las alertas:

- Selecciona las 3 alertas.
- Presiona **Ctrl + C**.

4. Copia los triggers de módulo (los que están al nivel del módulo, no dentro de bloques):

- WHEN-NEW-FORM-INSTANCE, KEY-EXIT, etc.

- Presiona **Ctrl + C**.

5. **Copia las Program Units** (si tienes procedimientos o funciones a nivel de módulo):

- Selecciona todas las del nodo **Program Units**.

- Presiona **Ctrl + C**.

6. **Copia las LOVs y Record Groups** (si tienes alguna, como la de puntos de venta):

- Presiona **Ctrl + C**.

Paso 4: Pega los objetos en el formulario destino

En el **Object Navigator** del formulario destino (el de AcseIX):

1. **Pega los bloques de datos:**

- Selecciona el nodo **Data Blocks**.

- Presiona **Ctrl + V**.

- Si algún nombre coincide con un bloque existente, Forms te preguntará cómo renombrar. Ponle un sufijo (ej. BLK_PV → BLK_PV_BLOQUEO).

2. **Pega las ventanas y canvases:**

- Selecciona **Windows** → **Ctrl + V**.

- Selecciona **Canvases** → **Ctrl + V**.

3. **Pega las alertas, triggers y program units.**

Paso 5: Ajusta los conflictos de nombres

Después de pegar, revisa:

1. **Canvas y ventanas nuevas:** abre el Property Palette de cada canvas pegado y verifica que su **Window** apunte a una ventana correcta. Si pegaste WIN_MAIN del origen pero el destino ya tenía otra WIN_MAIN, Forms te pedirá renombrar. Hazlo (ej. WIN_BLOQUEO).

2. **Ítems:** revisa que cada SET_ITEM_PROPERTY en los triggers apunte al nombre correcto del bloque/ítem después del pegado. Si renombraste BLK_PV a BLK_PV_BLOQUEO, actualiza TODAS las referencias en los triggers:

```
sql

-- Antes
SET_ITEM_PROPERTY('BLK_PV.NOMBRE_PV', ...);
```

-- Después

```
SET_ITEM_PROPERTY('BLK_PV_BLOQUEO.NOMBRE_PV', ...);
```

3.**Canvas de cada ítem:** verifica que cada ítem pegado tenga asignado el canvas correcto.

Paso 6: Fusiona el WHEN-NEW-FORM-INSTANCE

Esto es **crítico**. El form destino ya tiene su propio WHEN-NEW-FORM-INSTANCE de AcseIX. No puedes tener dos. Debes **fusionarlos**:

1. Abre el trigger WHEN-NEW-FORM-INSTANCE del form destino.
2. Copia el contenido de tu trigger original.
3. Pégalo al final del trigger existente, **antes del** END;.

Ejemplo:

sql

-- Contenido existente de AcseIX

BEGIN

-- Código de AcseIX (login, variables globales, permisos)

:GLOBAL.empresa := 'ACSEIX';

:GLOBAL.usuario_app := GET_APPLICATION_PROPERTY(USERNAME);

-- ... más código de AcseIX ...

-- ▼▼▼ TU CÓDIGO PEGADO AQUÍ ▼▼▼

:BLK_FILTRO.IT_ESTADO := 'TODOS';

:BLK_FILTRO.IT_SOLO_MORA := 'N';

GO_BLOCK('BLK_PV');

EXECUTE_QUERY;

-- ▲▲▲ FIN DE TU CÓDIGO ▲▲▲

END;

⚠ **No sobrescribas variables globales** que AcseIX ya usa. Si el sistema ya define :GLOBAL.usuario_app, NO la redefines; reutilízala.

Paso 7: Ajusta los CALL_FORM y OPEN_FORM del menú

Si copiaste el bloque de filtros y acciones, y quieres que este form sea accesible desde el menú de AcselX, el ítem del menú debe llamar al **form destino** (el que ahora contiene tu código), no al tuyo original:

```
sql
-- En el menú de AcselX
CALL_FORM('NOMBRE_FORM_ACSELX_DESTINO', NO_HIDE, NO_REPLACE, NO_QUERY_ONLY);
```

Paso 8: Compila y prueba

1. **Ctrl + K** para compilar el form destino.
2. **Ctrl + R** para ejecutarlo.
3. Verifica que el listado, los filtros, los botones y el popup funcionen.
4. Sube el `.fmx` al servidor de Forms (con WinSCP).
5. Prueba desde el menú de AcselX.

Cosas específicas a cuidar en AcselX

AcselX suele tener convenciones propias. Pregunta al equipo de TI o revisa el código existente:

Aspecto	Qué revisar
Variables globales	<code>:GLOBAL.usuario</code> , <code>:GLOBAL.empresa</code> , <code>:GLOBAL.sucursal</code> , etc. Usa las mismas, no crees las tuyas
Librerías adjuntas	El form destino tiene librerías (<code>.p11</code>) con funciones que DEBES usar. Adjúntalas si copias triggers
Bloque de control común	Muchos forms de AcselX tienen un <code>BLK_CONTROL</code> con botones estándar (Salir, Guardar, etc.)
Menú adjunto	No agregues menú a tu form; usa el del sistema
Nomenclatura	AcselX usa nombres como <code>BL_</code> , <code>TB_</code> , <code>IT_</code> . Respétalos

Aspecto	Qué revisar
Formato de fecha	Puede diferir del default de Forms. Revisa el WHEN-NEW-FORM-INSTANCE

Recomendación final

Antes de copiar todo, intenta arreglar el problema original. Pregunta al equipo de TI:

- 1.¿El `.fmx` está en el `FORMS_PATH` del servidor?
- 2.¿El `.mmx` del menú se recompiló después de agregar la opción?
- 3.¿Qué error exacto da cuando intentas abrirlo desde el menú? ¿Sale algún mensaje en el navegador o en los logs de WebLogic?

Si la respuesta es "es más fácil que copies todo al form que ya funciona", entonces hazlo, pero sigue estos pasos y **fusiona el** `WHEN-NEW-FORM-INSTANCE` con cuidado.

Reglas de oro al copiar:

- 1.**Nunca tengas dos** `WHEN-NEW-FORM-INSTANCE`. Fusiona.
- 2.**No sobreescribas variables globales de AcselX.**
- 3.**Verifica el nombre del bloque en cada** `SET_ITEM_PROPERTY` después de pegar.
- 4.**Revisa las librerías adjuntas** del form destino y no las elimines.
- 5.**Prueba en desarrollo antes de tocar producción.**

Si me dices cuál es el nombre del form que ya funciona en AcselX y qué error da al abrir el tuyo desde el menú, te puedo dar una solución más específica.

Entiendo perfectamente tu frustración. Crear bloques de datos manualmente en Oracle Forms Builder es totalmente posible y te da un control mucho mayor, pero hay detalles clave que, si se pasan por alto, impiden que el listado muestre datos. La razón más común por la que un bloque manual "no funciona" es que no se ha configurado correctamente su origen de datos o no se ha ejecutado la consulta explícitamente.

A continuación, te presento una guía detallada con los nombres de las propiedades en español (basados en las traducciones comunes de los manuales) y tres ejemplos completos para que puedas implementarlos sin depender del asistente.

Glosario de Propiedades Clave (Español)

Antes de empezar, aquí tienes los nombres de las propiedades que usarás una y otra vez. La "Paleta de Propiedades" es donde configurarás todo.

Propiedad (Inglés)	Nombre en Español (Común)	¿Para qué sirve?
Name	Nombre	Identificador único del objeto (bloque, ítem, etc.).
Database Data Block	Bloque de Datos de Base de Datos	Sí para bloques que leen/escriben en BD. No para bloques de control (botones, filtros).
Query Data Source Type	Tipo de Origen de Datos de Consulta	Define cómo se obtienen los datos: Tabla , Vista , Consulta FROM (FROM clause query), Procedimiento Almacenado .
Query Data Source Name	Nombre del Origen de Datos de Consulta	El nombre de la tabla/vista o la consulta SQL completa.
Query All Records	Consultar Todos los Registros	Si es sí , el bloque carga todos los registros al abrir el formulario.
Number of Records Displayed	Número de Registros Mostrados	Cuántas filas se ven a la vez en pantalla.
Update Allowed	Actualización Permitida	Si el bloque permite modificar datos.
Insert Allowed	Inserción Permitida	Si el bloque permite crear nuevos registros.
Delete Allowed	Eliminación Permitida	Si el bloque permite borrar registros.
Enforce Primary Key	Forzar Clave Primaria	Valida que no se dupliquen claves.
Navigation Style	Estilo de Navegación	Same Record (mismo registro) o Change Record (cambiar registro).

Propiedad (Inglés)	Nombre en Español (Común)	¿Para qué sirve?
Canvas	Lienzo	El "tablero" donde se dibujan los ítems.
Window	Ventana	La ventana del sistema que contiene al lienzo.
Prompt	Indicador (o Etiqueta)	El texto que se muestra al lado de un ítem.
Column Heading	Encabezado de Columna	El texto que se muestra arriba de una columna en un layout tabular.
Format Mask	Máscara de Formato	Formato de visualización para números y fechas.

Ejemplo 1: Bloque Manual basado en una TABLA

Este es el caso más directo. El bloque leerá y escribirá directamente en una tabla de la base de datos.

Paso 1: Crear el bloque manualmente

- 1.En el Navegador de Objetos, haz clic derecho sobre **Bloques de Datos** (Data Blocks) y selecciona **Crear** (Create).
- 2.En la ventana que aparece, elige "**Crear un nuevo bloque de datos manualmente**" (Build a new data block manually) y pulsa **Aceptar** (OK).

Paso 2: Configurar las propiedades del bloque

- 1.Selecciona el nuevo bloque (se llamará **BLOCK1**) y ve a la **Paleta de Propiedades**.
- 2.Cambia las siguientes propiedades:

Propiedad	Valor
Nombre	BLK_PV
Bloque de Datos de Base de Datos	Sí
Tipo de Origen de Datos de Consulta	Tabla
Nombre del Origen de Datos de Consulta	PUNTO_VENTA (el nombre de tu tabla)

Propiedad	Valor
Consultar Todos los Registros	No (lo haremos con un botón)
Número de Registros Mostrados	10
Forzar Clave Primaria	Sí

Paso 3: Crear los ítems

1. Haz clic derecho sobre el bloque BLK_PV y selecciona **Crear Ítem** (Create Item). Repite esto para cada columna que quieras mostrar.
2. Para cada ítem, configura su **Nombre** para que **coincida exactamente** con el nombre de la columna de la tabla. Por ejemplo, si tu tabla tiene una columna ID_PUNTO_VENTA, crea un ítem llamado ID_PUNTO_VENTA.
3. En la propiedad **Lienzo** (Canvas) de cada ítem, selecciona el lienzo donde quieras que aparezcan (ej. CVS_LISTA).
4. Para un layout tabular, acomoda los ítems en una fila.

Paso 4: Ejecutar la consulta

Para que los datos aparezcan, debes ejecutar la consulta. Puedes hacerlo al abrir el formulario o con un botón.

• **Trigger** WHEN-NEW-FORM-INSTANCE (a nivel de módulo):

```
plsql
```

```
BEGIN
```

```
GO_BLOCK('BLK_PV');
```

```
EXECUTE_QUERY; -- Esto carga los datos de la tabla
```

```
END;
```



Ejemplo 2: Bloque Manual basado en una VISTA

Es casi idéntico al anterior, pero el origen de datos es una vista. La ventaja es que puedes pre-calcular campos o hacer joins complejos.

Paso 1: Crear la vista en la base de datos

sql

```
CREATE OR REPLACE VIEW V_PV_RESUMEN AS
SELECT pv.id_punto_venta,
       pv.nombre_pv,
       pv.estado,
       NVL(SUM(f.saldo_pendiente), 0) AS total_mora
FROM punto_venta pv
LEFT JOIN cliente c ON c.id_punto_venta = pv.id_punto_venta
LEFT JOIN servicio s ON s.id_cliente = c.id_cliente
LEFT JOIN factura f ON f.id_servicio = s.id_servicio AND f.saldo_pendiente > 0
GROUP BY pv.id_punto_venta, pv.nombre_pv, pv.estado;
```

Paso 2: Crear el bloque manualmente

- 1.Crea un bloque manual como en el Ejemplo 1.
- 2.En la **Paleta de Propiedades** del bloque, configura:

Propiedad	Valor
Nombre	BLK_PV_RESUMEN
Bloque de Datos de Base de Datos	Sí
Tipo de Origen de Datos de Consulta	Vista
Nombre del Origen de Datos de Consulta	V_PV_RESUMEN

Paso 3: Crear los ítems

Crea ítems cuyos **Nombres** coincidan exactamente con los alias de las columnas de tu vista (ID_PUNTO_VENTA, NOMBRE_PV, ESTADO, TOTAL_MORA).

Paso 4: Ejecutar la consulta

Igual que en el Ejemplo 1, usa `EXECUTE_QUERY` para cargar los datos.

Ejemplo 3: Bloque Manual basado en una QUERY (FROM clause query)

Esta es la opción más flexible. Te permite escribir una sentencia `SELECT` personalizada (con `WHERE`, `ORDER BY`, etc.) directamente en el bloque.

Paso 1: Crear el bloque manualmente

1. Crea un bloque manual.
2. En la **Paleta de Propiedades**, configura:

Propiedad	Valor
Nombre	BLK_PV_QUERY
Bloque de Datos de Base de Datos	Sí
Tipo de Origen de Datos de Consulta	Consulta FROM (FROM clause query)
Nombre del Origen de Datos de Consulta	(SELECT id_punto_venta, nombre_pv, estado, direccion FROM punto_venta WHERE estado = 'ACTIVO')
Nota crucial: La consulta debe ir entre paréntesis . Si no, recibirás un error como FRM-41380.	

Paso 2: Crear los ítems

Crea ítems cuyos **Nombres** coincidan exactamente con los alias del `SELECT` (`ID_PUNTO_VENTA`, `NOMBRE_PV`, `ESTADO`, `DIRECCION`).

Paso 3: Ejecutar la consulta

Nuevamente, usa `EXECUTE_QUERY` para cargar los datos. Esta es la técnica recomendada para listados con filtros complejos sin necesidad de crear una vista.

Cómo Pasar Datos de un Listado a un Popup (Ventana Modal)

Este es un patrón muy común: tienes un listado (bloque `BLK_PV`) y quieres que al seleccionar una fila y presionar un botón "Ver Detalle", se abra una ventana más pequeña (modal) con los datos de esa fila.

Paso 1: Crear la ventana y el bloque del popup

1. **Ventana:** Crea una nueva ventana (`WIN_DETALLE`) y marca la propiedad **Modal** como **Sí**. Esto bloquea la ventana principal hasta que se cierre el popup.

- 2.**Lienzo:** Crea un lienzo (CVS_DETALLE) y asígnalo a WIN_DETALLE.
- 3.**Bloque:** Crea un **bloque de control** (Database Data Block = No) llamado BLK_DETALLE.
- 4.**Ítems:** Crea ítems en BLK_DETALLE que correspondan a los campos que quieres mostrar (ej. IT_ID_PV, IT_NOMBRE_PV, IT_DIAS_MORA). Asígnalos a CVS_DETALLE.

Paso 2: Crear el botón "Ver Detalle"

- 1.En tu bloque de botones principal, crea un botón BT_VER_DETALLE.
- 2.**Trigger** WHEN-BUTTON-PRESSED en BT_VER_DETALLE:

```
plsql

DECLARE
    v_id_pv NUMBER;
BEGIN
    -- 1. Verificar que hay una fila seleccionada
    v_id_pv := :BLK_PV.ID_PUNTO_VENTA;
    IF v_id_pv IS NULL THEN
        MESSAGE('Por favor, seleccione un registro del listado.');
```

```
        RAISE FORM_TRIGGER_FAILURE;
    END IF;

    -- 2. Pasar los datos al bloque del popup
    :BLK_DETALLE.IT_ID_PV      := :BLK_PV.ID_PUNTO_VENTA;
    :BLK_DETALLE.IT_NOMBRE_PV := :BLK_PV.NOMBRE_PV;
    :BLK_DETALLE.IT_DIAS_MORA := :BLK_PV.DIAS_MORA;

    -- 3. Mostrar la ventana modal
    SHOW_WINDOW('WIN_DETALLE');
    GO_ITEM('BLK_DETALLE.IT_ID_PV'); -- Mover el foco al popup
END;
```

Nota: :BLK_PV.ID_PUNTO_VENTA contiene el valor de la **fila actual** (la que está resaltada o donde está el cursor). Al presionar el botón, Forms ya ha actualizado ese valor a la fila que el usuario seleccionó.

Paso 3: Cerrar el popup

En el popup, crea un botón BT_CERRAR con el trigger WHEN-BUTTON-PRESSED:

```
plsql

BEGIN
    HIDE_WINDOW('WIN_DETALLE');
    GO_BLOCK('BLK_PV'); -- Devolver el foco al listado principal
```

END;

Errores Comunes por los que un Bloque Manual "No Muestra Datos"

Si seguiste los pasos y aún no ves datos, revisa estos puntos:

1. **No se ejecutó EXECUTE_QUERY**: El bloque no carga datos automáticamente. Debes llamar a EXECUTE_QUERY (ya sea en WHEN-NEW-FORM-INSTANCE o en un botón "Buscar").
2. **Nombres de ítems incorrectos**: Los nombres de los ítems **deben coincidir exactamente** con los nombres de las columnas en el SELECT (o con los alias). Si el SELECT devuelve ID_PUNTO_VENTA y tu ítem se llama ID_PV, no se mapeará.
3. **El bloque no está en un lienzo (Canvas)**: Todos los ítems deben tener asignado un lienzo en su propiedad **Lienzo** (Canvas). Si no, no se muestran.
4. **Query Data Source Name mal escrito**: Verifica que el nombre de la tabla/vista sea correcto o que la consulta esté bien escrita y entre paréntesis.
5. **El bloque es de "control" pero debería ser de "base de datos"**: Asegúrate de que **Bloque de Datos de Base de Datos** esté en **Sí** si quieres leer/escribir en la BD.
6. **Faltan datos en la tabla**: Si la tabla está vacía o no has hecho COMMIT después de insertar datos desde SQL*Plus, el bloque no encontrará nada.
7. **El bloque no tiene ítems navegables**: Para que EXECUTE_QUERY funcione, el bloque debe tener al menos un ítem con **Navegable** = **Sí**.

Con estos ejemplos y la lista de verificación, deberías poder crear y depurar tus listados manuales sin depender del asistente. La clave está en configurar correctamente las propiedades del bloque y asegurarte de ejecutar la consulta.

luego, vamos a resolver esos tres puntos con ejemplos prácticos y probados, para que no tengas que adivinar nada. La clave está en configurar bien las propiedades del bloque de datos y usar los triggers correctos.

1. Botón de Buscar con Filtro por Nombre

La forma más limpia y profesional es usar un **bloque de control** para el campo de búsqueda y un botón que modifique la consulta del bloque de datos.

Paso a paso:

1. Crea el campo de búsqueda y el botón:

- Crea un bloque de control (por ejemplo, BLK_BUSQUEDA). Recuerda que en la Paleta de Propiedades debes poner **Bloque de Datos de Base de Datos** = No.
- Dentro de este bloque, crea un ítem de texto llamado IT_BUSCAR_NOMBRE. Asígnale el **Lienzo** (Canvas) donde quieras que aparezca.
- Crea un botón llamado BT_BUSCAR. Asígnale el mismo lienzo.

2. Asigna el trigger al botón:

- Haz clic derecho sobre el botón BT_BUSCAR y selecciona **Crear Disparador** (Create Trigger) -> **WHEN-BUTTON-PRESSED**.

3. Escribe el código del trigger:

Este código construye dinámicamente la cláusula **WHERE** y la aplica al bloque del listado antes de volver a ejecutar la consulta. El bloque del listado, en este ejemplo, se llama BLK_PV.

```
plsql
```

```
DECLARE
```

```
    v_where VARCHAR2(4000);
```

```
BEGIN
```

```
    -- 1. Construye la cláusula WHERE dinámicamente
```

```
    IF :BLK_BUSQUEDA.IT_BUSCAR_NOMBRE IS NOT NULL THEN
```

```
        v_where := 'UPPER(nombre_pv) LIKE ''' || UPPER(:BLK_BUSQUEDA.IT_BUSCAR_NOMBRE) ||
```

```
        '%''';
```

```
    END IF;
```

```
    -- 2. Aplica el WHERE al bloque del listado
```

```
    SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
```

```
    -- 3. Mueve el foco al bloque y ejecuta la consulta
```

```
    GO_BLOCK('BLK_PV');
```

```
    EXECUTE_QUERY;
```

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
MESSAGE('Error al buscar: ' || SQLERRM);  
RAISE FORM_TRIGGER_FAILURE;
```

```
END;
```

Explicación:

- `SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where)`: Esta es la clave. Le dice a Oracle Forms que use la cadena `v_where` como filtro la próxima vez que se consulte el bloque.
- `GO_BLOCK('BLK_PV')`: Mueve el cursor al bloque de resultados. Es obligatorio porque `EXECUTE_QUERY` solo funciona en el bloque actual.
- `EXECUTE_QUERY`: Vuelve a ejecutar la consulta, trayendo solo los registros que cumplen con el filtro.

Para limpiar el filtro: Puedes añadir un botón "Limpiar" que ejecute `SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, '')`; y luego `EXECUTE_QUERY`;

2. Paginación (Mostrar de 10 en 10) y Barra de Desplazamiento

Aquí hay que diferenciar dos conceptos: **cuántos registros se muestran a la vez** y **cuántos se traen de la base de datos**.

Propiedades del Bloque de Datos (en la Paleta de Propiedades, sección "Registros"):

- `Número de Registros Mostrados (Number of Records Displayed)`: Es el número de filas que se ven en pantalla simultáneamente. Para mostrar de 10 en 10, pon este valor en **10**.
- `Tamaño del Array de Consulta (Query Array Size)`: Es el número de registros que Forms trae de la base de datos en cada viaje de red. Si lo pones en **0** (el valor por defecto), traerá la misma cantidad que "Número de Registros Mostrados" (es decir, 10). Si quieres traer más para que la navegación sea más rápida (por ejemplo, 50), puedes aumentar este valor.
- `Consultar Todos los Registros (Query All Records)`: Si lo pones en **Sí**, Forms traerá TODOS los registros de una sola vez, ignorando el "Tamaño del Array de Consulta". **NO lo pongas en Sí** si quieres paginación; déjalo en **No** para que Forms traiga los registros por lotes.

Cómo mostrar la barra de desplazamiento vertical:

- En la Paleta de Propiedades del **bloque de datos**, busca la propiedad `Mostrar Barra de Desplazamiento (Show Scroll Bar)` y ponla en **Sí (Yes)**.

- Justo debajo, verás **Orientación de la Barra de Desplazamiento (Scroll Bar Orientation)**. Selecciona **Vertical**.
- Con esto, Forms dibujará automáticamente una barra vertical al lado del bloque en el lienzo. El usuario podrá usarla para navegar entre los registros que no se ven en pantalla.

Nota sobre el "scroll" del lienzo: Si lo que quieres es que toda la pantalla tenga scroll (porque hay muchos campos), la solución es usar un **Lienzo Apilado (Stacked Canvas)** y poner la barra de desplazamiento en la **Ventana (Window)**. Para el scroll de un listado de datos, la barra del bloque es la correcta.

3. Ejemplo Práctico: Cómo Mostrar el Detalle en un Popup

Ya tienes el listado con búsqueda y scroll. Ahora, el botón "Ver Detalle" que abre una ventana modal con la información de la fila seleccionada.

Paso a paso:

1. Crea la ventana modal y su lienzo:

- Crea una ventana **WIN_DETALLE** y marca su propiedad **Modal** como **Sí**. Esto bloquea la ventana principal hasta que se cierre.
- Crea un lienzo **CVS_DETALLE** y asígnalo a **WIN_DETALLE**.

2. Crea el bloque de control y sus ítems:

- Crea un bloque **BLK_DETALLE** (Bloque de Datos de Base de Datos = **No**).
- Dentro, crea los ítems que quieras mostrar (ej. **IT_ID_PV**, **IT_NOMBRE_PV**, **IT_DIAS_MORA**) y asígnalos al lienzo **CVS_DETALLE**.

3. Crea el botón y su trigger en el listado:

- En tu bloque de botones del listado, crea **BT_VER_DETALLE**.
- Añade un trigger **WHEN-BUTTON-PRESSED** con este código:

```
plsql
```

```
DECLARE
```

```
    v_id_pv NUMBER;
```

```
BEGIN
```

```
    -- 1. Verifica que haya una fila seleccionada
```

```
    v_id_pv := :BLK_PV.ID_PUNTO_VENTA; -- Asume que el ítem del ID se llama así
```

```
    IF v_id_pv IS NULL THEN
```

```

        MESSAGE('Por favor, seleccione un registro del listado.');
```

RAISE FORM_TRIGGER_FAILURE;

```

END IF;
```

-- 2. Pasa los datos de la fila actual a los ítems del popup

```

:BLK_DETALLE.IT_ID_PV      := :BLK_PV.ID_PUNTO_VENTA;
:BLK_DETALLE.IT_NOMBRE_PV := :BLK_PV.NOMBRE_PV;
:BLK_DETALLE.IT_DIAS_MORA := :BLK_PV.DIAS_MORA;
```

-- 3. Muestra la ventana modal

```
SHOW_WINDOW('WIN_DETALLE');
```

```
END;
```

Clave: `:BLK_PV.NOMBRE_PV` contiene el valor de la **fila actual**, que es la que el usuario tiene seleccionada con el cursor.

4.Crea el botón de cerrar en el popup:

- Dentro del bloque `BLK_DETALLE`, crea un botón `BT_CERRAR`.
- Su trigger `WHEN-BUTTON-PRESSED` será:

```
plsql
```

```
BEGIN
```

```

    HIDE_WINDOW('WIN_DETALLE');
    GO_BLOCK('BLK_PV'); -- Devuelve el foco al listado
```

```
END;
```

Resumen y Buenas Prácticas

- Para buscar:** Usa un bloque de control y `SET_BLOCK_PROPERTY(..., DEFAULT_WHERE, ...)` seguido de `EXECUTE_QUERY`.
- Para paginar:** Configura `Número de Registros Mostrados` en 10 y `Tamaño del Array de Consulta` en un valor mayor (o 0) para controlar cuántos registros se traen de la BD a la vez.
- Para el scroll:** Activa `Mostrar Barra de Desplazamiento` en el bloque y elige `Vertical`.
- Para el popup:** Usa una ventana **Modal**, un bloque de control y pasa los datos del ítem actual del listado a los ítems del popup directamente en el trigger del botón.

Con estos tres patrones, puedes construir interfaces de usuario profesionales en Oracle Forms sin depender de los asistentes. Si algún trigger te da un error específico, puedes consultarlo y lo revisamos.

